Convolutional Neural Networks

Unit 3 — Foundations, Operations, Visualisation & Architectures

1 Foundations of CNNs

1.1 Why CNNs Exist: Limitations of MLPs on Image Data

A Multi-Layer Perceptron (MLP) treats every input as a flat vector of independent values. When applied to images, this design breaks down in three distinct, compounding ways.

1.1.1 Parameter Explosion

An MLP connects every input pixel to every neuron in the first hidden layer. For a modest $224 \times 224 \times 3$ colour image, the input has 150,528 values. A single hidden layer with 4,096 neurons would require $150{,}528 \times 4{,}096 \approx 617$ million weights—before any other layer is added. Training such a network is computationally infeasible, and the model will overfit severely on any realistic dataset size.

1.1.2 No Spatial Awareness

Flattening an image to a 1-D vector destroys all spatial structure. Pixels that are neighbours in the image become arbitrary entries in the vector; their proximity carries no meaning to the MLP. A horizontal edge in the top-left corner and the same edge in the bottom-right corner produce completely different activations and must be learned as separate patterns.

1.1.3 No Translation Invariance

An MLP has no mechanism to recognise that a cat in the upper-left of an image is the same kind of object as a cat in the centre. Each spatial position is handled by a unique set of weights, so the network must learn the same concept multiple times for every position it might appear in. This is both wasteful and impossible to achieve with finite training data.

> **Common misconception:** People often say MLPs "cannot handle images." They can, technically—the architecture is not wrong, it is just catastrophically inefficient and generalises poorly. The CNN is not a necessity; it is a principled solution to known structural problems.

1.2 The Three Core Ideas CNNs Are Built On

1.2.1 Local Receptive Fields

Each neuron in a convolutional layer connects to a small, spatially contiguous region of the input (e.g. 3×3 or 5×5 pixels), called its *receptive field*. This exploits the fact that meaningful visual patterns—edges, corners, textures—are inherently local. A neuron does not need to see the entire image to detect a horizontal edge; it only needs a small patch.

> **Receptive Field (local):** The specific region of the input space that a single neuron responds to. In early layers this is small (a few pixels); in deeper layers it grows to

encompass large portions of the image.

1.2.2 Parameter Sharing

A single filter (kernel) is applied at every spatial location of the input. All positions share the same weights. This reduces the parameter count from millions to tens or hundreds per filter, and encodes the assumption that a feature detector useful in one region is useful everywhere. A filter that detects a vertical edge will detect it correctly regardless of where in the image it appears.

1.2.3 Spatial Hierarchy

CNNs stack multiple convolutional layers. Early layers detect simple features (edges, blobs); subsequent layers combine those into more complex structures (corners, textures); deeper layers assemble those into high-level semantic concepts (eyes, wheels, faces). This hierarchical composition mirrors how biological visual systems process scenes and allows the network to build up complex representations from simple primitives without needing to explicitly hand-engineer any features.

Intuition: Think of spatial hierarchy like reading. Individual strokes form letters; letters form words; words form sentences. You don't read a sentence as 500 individual pen strokes—your brain builds up meaning in stages. A CNN does exactly this with pixels.

1.3 Overall CNN Architecture

The canonical CNN pipeline is:

$$\begin{aligned}\text{Input} \;\rightarrow\; [\text{Conv} \rightarrow \text{Activation}]^{\times n} \;\rightarrow\; \text{Pooling} \;\rightarrow\; \cdots \\ \rightarrow\; \text{Flatten} \;\rightarrow\; [\text{FC} \rightarrow \text{Activation}]^{\times m} \;\rightarrow\; \text{Output}\end{aligned}$$

- **Convolutional layers:** Extract spatial features via learned filters. Output is a 3-D feature map (height $\times$ width $\times$ depth).
- **Activation (ReLU):** Introduces non-linearity. Applied element-wise after each convolution.
- **Pooling layers:** Downsample the spatial dimensions, reducing computation and providing local translation invariance.
- **Flatten:** Converts the 3-D feature map into a 1-D vector for fully connected processing.
- **Fully connected (FC) layers:** Perform classification or regression using the high-level features assembled by the conv stack.
- **Output layer:** Softmax for classification (probability over classes); linear for regression.

1.4 CNNs vs. MLPs: Structural and Capability Differences

Property	MLP	CNN
Connectivity	Fully connected (every neuron to every input)	Locally connected (each neuron to a small patch)
Parameter sharing	None—each connection has a unique weight	Yes—one filter slides over all positions
Spatial awareness	Destroyed by flattening	Preserved through 3-D feature maps
Translation invariance	None	Approximate (via shared weights + pooling)
Parameter count	Extremely large for image inputs	Manageable; grows with depth, not input size
Inductive biases	None	Locality, stationarity, compositionality
Typical use	Tabular data, small fixed-size inputs	Images, audio spectrograms, spatial sequences

2 Convolution, Striding, Padding, and Pooling

2.1 The Convolution Operation

Convolution (in CNNs): A filter (kernel) of size $F \times F$ is slid over a 2-D input. At each position, an element-wise multiplication is performed between the filter weights and the corresponding input patch, and the results are summed to produce a single scalar output. This scalar becomes one entry in the output *feature map*. The operation is repeated across all positions.

Formally, for a 2-D input I and a filter K of size $F \times F$, the output feature map S at position (i, j) is:

$$S(i,j) = \sum_{m=0}^{F-1} \sum_{n=0}^{F-1} I(i+m,\ j+n) \cdot K(m,n)$$

In practice, CNNs use *cross-correlation* (no filter flip) rather than true convolution, but the term "convolution" is used throughout the field by convention.

Intuition: Place a small magnifying template (the filter) over the image. Multiply every pixel under the template by the corresponding number in the template, sum everything up, write down the result. Slide the template one step to the right. Repeat until you've covered the entire image. Each result is a measure of "how strongly this filter pattern is present at this location."

2.1.1 Output Size Formula (no padding, stride 1)

$$W_{\text{out}} = W_{\text{in}} - F + 1 \qquad H_{\text{out}} = H_{\text{in}} - F + 1$$

A 32×32 input convolved with a 5×5 filter produces a 28×28 output.

2.2 Stride

Stride (S): The number of pixels the filter moves at each step. A stride of 1 moves one pixel at a time (dense sampling). A stride of 2 skips every other position, producing a subsampled output.

2.2.1 Output Size Formula with Stride

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - F}{S} \right\rfloor + 1$$

2.2.2 Trade-off

Increasing stride reduces output spatial resolution: the network samples fewer positions and the feature map shrinks. This reduces computation and memory, but discards spatial information between sampled locations. Strided convolutions are increasingly used instead of pooling as an explicit downsampling strategy because the filter weights remain learnable—the network decides *how* to downsample, not just *when*.

Striding vs. Pooling for downsampling: Both reduce spatial size, but strided convolution learns its downsampling behaviour from data, while pooling uses a fixed rule (max or average). Striding has more parameters; pooling has none. Modern architectures (ResNet, etc.) often prefer strided convolution over pooling for this reason.

2.3 Padding

Padding: Adding extra values (typically zeros) around the border of the input before applying convolution, so that the filter can be centred on edge and corner pixels and the output size can be controlled.

2.3.1 Valid Padding

No padding is added. The filter only slides where it fully overlaps the input. Output is strictly smaller than the input. Edge pixels contribute to fewer output values than central pixels—they are under-represented in the feature map.

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - F}{S} \right\rfloor + 1$$

2.3.2 Same Padding

Padding P is chosen so that the output has the same spatial size as the input (when $S = 1$). For a filter of size F:

$$P = \left\lfloor \frac{F - 1}{2} \right\rfloor$$

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} + 2P - F}{S} \right\rfloor + 1$$

Common misconception: "Same padding preserves image size." This is only true when stride = 1. With stride > 1, even same padding will shrink the output. The word "same" refers to the formula being configured to match input size at stride 1—not a universal guarantee of size preservation.

2.3.3 Why Padding Matters

Without padding, feature maps shrink at every conv layer, rapidly collapsing to a tiny spatial footprint. Networks become shallow by necessity. Padding lets the designer control the size trajectory of feature maps independently of filter size, enabling much deeper architectures.

2.4 Multiple Filters and the Depth Dimension

A single filter produces a single 2-D feature map. To detect multiple different features simultaneously, a conv layer applies D filters in parallel, producing a stack of D feature maps. This stack has shape $H_{\text{out}} \times W_{\text{out}} \times D$—the depth dimension represents the number of distinct feature detectors active at that layer.

Each filter is a full-depth tensor: if the input has C channels (e.g. 3 for RGB), each filter has shape $F \times F \times C$, so it operates on all channels at once and still produces a single 2-D output map. With D filters, the output has D channels.

Intuition: Think of each filter as a specialised detector. One filter lights up for vertical edges, another for horizontal edges, another for blue blobs. The depth of the output is the count of detectors running in parallel. After many layers, these "detectors" become abstract—not edges, but "face-like-things" or "wheel-like-things."

2.5 Pooling

Pooling: A fixed, non-learnable operation that reduces the spatial dimensions of a feature map by summarising values within a local window. The two standard variants are *max pooling* (take the maximum value in the window) and *average pooling* (take the mean of all values in the window).

2.5.1 Max Pooling

For a pooling window of size $P \times P$ with stride S:

$$W_{\text{out}} = \left\lfloor \frac{W_{\text{in}} - P}{S} \right\rfloor + 1$$

The output at each position is the maximum activation in the corresponding input window. Max pooling retains the strongest detected feature, discarding exact position within the window. This provides *local translation invariance*: a feature shifted by a few pixels within the pooling window produces the same output.

2.5.2 Average Pooling

The output at each position is the mean of all values in the window. Average pooling produces smoother feature maps and is less aggressive about discarding spatial detail. It is less common in intermediate layers but used in some architectures (e.g. Global Average

Pooling before the classifier, which replaces FC layers entirely).

2.5.3 Pooling: Advantages and Disadvantages

Advantages:

- Reduces spatial dimensions → reduces parameter count and computation in subsequent layers.
- Provides local translation invariance.
- No learnable parameters—computationally cheap with no risk of overfitting its own weights.
- Acts as a mild form of regularisation by discarding precise positional information.

Disadvantages:

- Destroys spatial information; exact position of detected features is lost.
- Fixed rule—cannot adapt its behaviour based on the data.
- Max pooling is not differentiable at tie positions; subgradients are used in practice.
- Some modern architectures argue that pooling is harmful and replace it entirely with strided convolutions (e.g. "all-convolutional" networks).

Pooling vs. Striding confusion: Both downsample. Pooling acts *after* a convolution, summarising the feature map with a fixed operation. Striding acts *during* convolution by simply skipping positions. Striding is learnable; pooling is not. They are not the same operation, even though the output size formula is structurally identical.

2.6 Comparison: Convolution vs. Pooling vs. Striding as Downsampling Strategies

Property	**Conv (stride 1)**	**Pooling**	**Strided Conv**
Primary purpose	Feature extraction	Spatial reduction	Feature extraction + reduction
Learnable	Yes	No	Yes
Reduces spatial size	Only without same padding	Yes	Yes
Translation invariance	Via weight sharing	Via fixed window summary	Via weight sharing
Spatial info preserved	Mostly	Partially (max) / poorly (avg)	Partially
Computation cost	Moderate	Very low	Moderate
Used when	Extracting features at same scale	Cheap downsampling	Downsampling with learned behaviour
Modern preference	Always	Declining	Increasingly preferred

3 Understanding and Visualising CNNs

3.1 What Each Layer Learns

A CNN does not learn abstract features uniformly across its depth. There is a consistent empirical pattern:

- **Early layers (conv1, conv2):** Detect low-level features—oriented edges at various angles, colour blobs, simple gradients, and on/off centre-surround patterns. These filters strongly resemble Gabor wavelets found in the mammalian primary visual cortex (V1). The features are local, small-scale, and largely interpretable by inspection.
- **Middle layers:** Combine low-level features into textures, corners, junctions, and simple shapes. A middle-layer filter might activate maximally on a grid texture or a curved contour—patterns that are not present in any single pixel but arise from the composition of edge detectors.
- **Deep layers (near the classifier):** Assemble mid-level structures into high-level semantic parts. In a network trained on ImageNet, these filters respond to dog faces, car wheels, eye-like structures, or architectural details. The representations here are both abstract and class-discriminative.

Intuition: Early layers ask "is there a vertical edge here?" Middle layers ask "is there a corner made of two edges?" Deep layers ask "does this region look like a dog's snout?" Each layer's answer is built from the previous layer's answers.

3.2 Receptive Field and Why It Grows with Depth

Effective Receptive Field: The region of the original input image that influences the value of a given neuron at any depth. Neurons in early layers have small receptive fields (a few pixels); neurons in deep layers have large ones (potentially spanning most of the image).

With a stack of 3×3 filters, each additional conv layer increases the effective receptive field diameter by 2 pixels in each direction. After L layers of 3×3 convolutions with stride 1, the theoretical receptive field diameter is $2L+1$. Pooling and strided convolutions multiply this growth. This is why depth matters: a single conv layer with a large filter could cover the same receptive field, but it would have far more parameters and lack the compositional, non-linear structure that makes deep networks expressive.

Theoretical vs. effective receptive field: The theoretical receptive field is the maximum area that *could* influence a neuron. The effective receptive field is smaller—not all input pixels contribute equally. Central pixels have a stronger gradient path to the output than peripheral ones. The gap matters in interpretability: a neuron's effective receptive field can be much smaller than its theoretical one.

3.3 Visualisation Techniques

3.3.1 Filter Visualisation

The simplest approach: directly display the learned filter weights as an image. For first-layer filters operating on raw pixels, this is interpretable—you see oriented bars, colour

patches, or opponent-colour detectors. For deeper layers, this approach is not meaningful because the filters operate on abstract feature map channels, not pixel values.

3.3.2 Activation Maps

Feed a specific input image through the network and record the feature map values at a chosen layer. The resulting 2-D maps show where in the image a particular filter "fired." High values indicate spatial locations where the pattern the filter detects is present. These maps can be overlaid on the original image to show which regions activated a particular detector.

3.3.3 Saliency Maps

A gradient-based technique: compute the gradient of the class score (before softmax) with respect to the input pixels. Pixels with large-magnitude gradients are those whose small perturbation would most affect the class prediction—they are the pixels the network finds most informative for its decision.

$$\text{Saliency}(x) = \left| \frac{\partial S_c}{\partial x} \right|$$

where S_c is the score for class c and x is the input image. The saliency map highlights which input pixels the network is attending to.

3.3.4 Grad-CAM (Gradient-weighted Class Activation Mapping)

Grad-CAM is a more spatially accurate technique that operates on the *feature maps of the last convolutional layer*. The gradient of the class score with respect to each filter's activation map is computed and global-average-pooled to yield an importance weight α_k^c for filter k:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial S_c}{\partial A_{ij}^k}$$

The Grad-CAM heatmap is then formed as a weighted combination of the activation maps, passed through ReLU to retain only positive influences:

$$L_{\text{Grad-CAM}}^c = \text{ReLU}\left(\sum_k \alpha_k^c A^k \right)$$

The result is upsampled to the input resolution and overlaid on the image. Grad-CAM is class-discriminative (different classes produce different heatmaps), requires no architectural changes, and is more spatially precise than raw saliency maps.

Intuition: Grad-CAM asks: "For the last conv layer, which feature maps mattered most for predicting 'cat'? And where in the image were those feature maps most active?" The overlap of "mattered" and "where" produces a heatmap pointing to the image region the network relied on.

3.4 Why CNNs Generalise Well: Inductive Biases

An inductive bias is an assumption baked into the architecture that constrains the space of functions the model can learn. CNNs have three strong inductive biases that match the

statistical structure of natural images:

- **Locality:** Nearby pixels are more correlated than distant ones. Restricting connections to local patches exploits this prior without needing to learn it from data.
- **Translation stationarity:** The statistical properties of natural images are approximately the same everywhere—an edge looks like an edge regardless of its position. Parameter sharing directly encodes this assumption.
- **Compositionality:** Complex patterns are built from simpler ones. Hierarchical stacking of layers mirrors this compositional structure.

Because these biases match reality, CNNs require far fewer training examples to generalise than MLPs. The architecture itself rules out a large class of irrelevant functions, focusing capacity on the subset relevant to image understanding.

3.5 Common Failure Modes

3.5.1 Adversarial Examples

Small, human-imperceptible perturbations added to an image can cause a CNN to misclassify with high confidence. These perturbations exploit the linearity of the decision boundary in high-dimensional pixel space. A panda image with structured noise added can be classified as a gibbon with 99% confidence, while humans see only a panda. CNNs have not learned truly robust semantic features—they are sensitive to non-semantic directions in the input space.

3.5.2 Texture Bias

CNNs, especially those trained on ImageNet, exhibit a strong *texture bias*: when shape and texture cues conflict (e.g. a cat silhouette filled with elephant skin texture), networks classify based on texture rather than shape. Humans rely primarily on shape. This indicates that CNNs can learn texture statistics as a shortcut rather than the underlying geometry of objects.

3.5.3 Sensitivity to Rotation and Scale

Standard CNNs have no built-in invariance to rotation or large scale changes. A digit rotated by 90°, or an object photographed at an unusual scale, can fool a standard CNN even when it classifies the canonical orientation correctly. Data augmentation (random rotations, crops, flips during training) partially mitigates this, at the cost of longer training time.

Translation invariance is not free: Pooling provides only *local* translation invariance—small shifts within a pooling window produce the same output. Large translations are not handled. A CNN trained on centred faces will fail on faces appearing in corners. Large-scale translation invariance must come from data augmentation, not from the architecture alone.

4 Classic CNN Architectures

4.1 LeNet-5 (1998)

4.1.1 Historical Context

Proposed by Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner at Bell Labs, LeNet-5 was designed for handwritten digit recognition and deployed commercially to read cheque amounts. It was the first architecture to demonstrate that convolutional networks could be trained end-to-end with backpropagation on real-world tasks at scale.

4.1.2 Architecture (Layer by Layer)

Input: $32 \times 32 \times 1$ (grayscale, padded from 28×28 MNIST digit)

1. **C1 — Conv:** 6 filters, 5×5, stride 1, no padding. Output: $28 \times 28 \times 6$. Activation: tanh.
2. **S2 — Average Pooling:** 2×2, stride 2. Output: $14 \times 14 \times 6$.
3. **C3 — Conv:** 16 filters, 5×5, stride 1. Output: $10 \times 10 \times 16$. (Sparse connectivity: not all 6 input maps feed into all 16 filters, by design.) Activation: tanh.
4. **S4 — Average Pooling:** 2×2, stride 2. Output: $5 \times 5 \times 16$.
5. **C5 — Conv:** 120 filters, 5×5. Output: $1 \times 1 \times 120$ (functionally a FC layer). Activation: tanh.
6. **F6 — Fully Connected:** 84 neurons. Activation: tanh.
7. **Output:** 10 neurons (one per digit class). Gaussian/RBF units in the original paper; softmax in modern re-implementations.

Total parameters: approximately 60,000.

4.1.3 Key Innovations

- First practical demonstration of end-to-end backpropagation-trained convolutional networks on a real task.
- Established the conv–pool–conv–pool–FC–output template used by virtually every subsequent classification architecture.
- Demonstrated parameter sharing and local connectivity as viable replacements for full connectivity.

4.1.4 Limitations

- Too shallow and narrow for complex, high-resolution, multi-class natural images.
- Sigmoid/tanh activations saturate for large inputs, causing vanishing gradients in deeper networks.
- No dropout; prone to overfitting on larger, more varied datasets.
- Input restricted to 32×32 grayscale; cannot handle the diversity of natural images.
- Average pooling is less sharp than max pooling for detecting strong local features.

4.2 AlexNet (2012)

4.2.1 Historical Context

AlexNet, designed by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, won the 2012 ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with a top-5 error of 15.3%, compared to 26.2% for the second-place entry. This gap was so large it effectively ended the era of hand-crafted feature engineering and launched the modern deep learning revolution.

4.2.2 Architecture (Layer by Layer)

Input: $224 \times 224 \times 3$ (RGB)

1. **Conv1:** 96 filters, 11×11, stride 4. Output: $55 \times 55 \times 96$. ReLU. LRN.
2. **MaxPool1:** 3×3, stride 2. Output: $27 \times 27 \times 96$.
3. **Conv2:** 256 filters, 5×5, same padding. Output: $27 \times 27 \times 256$. ReLU. LRN.
4. **MaxPool2:** 3×3, stride 2. Output: $13 \times 13 \times 256$.
5. **Conv3:** 384 filters, 3×3, same padding. Output: $13 \times 13 \times 384$. ReLU.
6. **Conv4:** 384 filters, 3×3, same padding. Output: $13 \times 13 \times 384$. ReLU.
7. **Conv5:** 256 filters, 3×3, same padding. Output: $13 \times 13 \times 256$. ReLU.
8. **MaxPool3:** 3×3, stride 2. Output: $6 \times 6 \times 256$.
9. **FC6:** 4096 neurons. ReLU. Dropout ($p = 0.5$).
10. **FC7:** 4096 neurons. ReLU. Dropout ($p = 0.5$).
11. **Output:** 1000 neurons. Softmax.

Total parameters: approximately 60 million.

4.2.3 What Made It Revolutionary

- **ReLU activations:** Replaced sigmoid/tanh. ReLU does not saturate for positive inputs, dramatically accelerating gradient flow and convergence (reported 6× faster than tanh in the original paper).
- **GPU training:** Trained across two NVIDIA GTX 580 GPUs (3 GB VRAM each), with the model split between them. Demonstrated that consumer GPU hardware could make deep networks tractable on large datasets.
- **Dropout:** Applied in FC layers with $p = 0.5$. Reduced co-adaptation of neurons and significantly reduced overfitting; now a standard regularisation technique.
- **Data augmentation:** Random 224×224 crops from 256×256 images, horizontal flips, and PCA-based colour jittering. Artificially expanded the effective dataset size and dramatically reduced overfitting.
- **Local Response Normalisation (LRN):** A lateral inhibition mechanism inspired by neuroscience; largely abandoned in subsequent work in favour of Batch Normalisation.
- **Scale:** 5 conv layers + 3 FC layers—far deeper than anything trained end-to-end before.

Why ReLU changed everything: Sigmoid outputs values in $(0, 1)$; for inputs far from zero, the gradient approaches zero (saturation). Stacking many sigmoid layers means gradients shrink exponentially—the vanishing gradient problem. ReLU outputs 0 for negative input and the input itself for positive input. Gradient is either 0 or 1 for any single unit—no saturation, no exponential shrinkage, stable gradient flow in the positive regime.

4.2.4 Limitations

- Large 11×11 filter in Conv1 is computationally expensive and covers a coarsely sampled receptive field.
- Very large FC layers (4096×4096) account for the majority of parameters with limited spatial reasoning capacity.
- LRN is now understood to contribute marginally and is rarely used.
- Architectural choices are irregular (varying filter sizes, uneven layer widths)—no clear design philosophy emerges.

4.3 VGGNet (2014)

4.3.1 Historical Context

Proposed by Karen Simonyan and Andrew Zisserman at the Visual Geometry Group, Oxford (hence VGG). VGG-16 achieved second place in ILSVRC 2014 but proved more influential than the winner (GoogLeNet) because of its architectural clarity and transferability. Its core thesis—that depth alone, using exclusively 3×3 convolutions, drives performance—became a foundational design principle.

4.3.2 Architecture Philosophy

VGGNet discards heterogeneous filter sizes in favour of a single rule: **use only 3×3 convolutions with stride 1 and same padding**. The network is organised into blocks of 2–4 consecutive conv layers followed by a 2×2 max pool with stride 2. Depth is increased by adding more blocks. All FC layers use 4096 units.

VGG-16 (13 conv + 3 FC):

1. **Block 1:** Conv-64, Conv-64 $\rightarrow$ MaxPool. Output: $112 \times 112 \times 64$.
2. **Block 2:** Conv-128, Conv-128 $\rightarrow$ MaxPool. Output: $56 \times 56 \times 128$.
3. **Block 3:** Conv-256, Conv-256, Conv-256 $\rightarrow$ MaxPool. Output: $28 \times 28 \times 256$.
4. **Block 4:** Conv-512, Conv-512, Conv-512 $\rightarrow$ MaxPool. Output: $14 \times 14 \times 512$.
5. **Block 5:** Conv-512, Conv-512, Conv-512 $\rightarrow$ MaxPool. Output: $7 \times 7 \times 512$.
6. FC: $4096 \rightarrow 4096 \rightarrow 1000$ (softmax).

Total parameters: approximately 138 million (VGG-16).

4.3.3 Why Small Filters Work Better Than Large Ones

Two stacked 3×3 filters have an effective receptive field of 5×5. Three stacked 3×3 filters cover 7×7. The stack is strictly superior to a single large filter for two reasons:

- **Fewer parameters:** Two 3×3 layers have $2 \times (9C^2) = 18C^2$ parameters (for C channels). One 5×5 layer has $25C^2$—39% more, for the same theoretical receptive field. Three 3×3 layers ($27C^2$) have fewer parameters than one 7×7 layer ($49C^2$).
- **More non-linearity:** Each 3×3 layer is followed by a ReLU. Three layers with two intermediate non-linearities are strictly more expressive than one layer with a single non-linearity at the end, for the same receptive field.

Intuition: Instead of one big question ("does this 7×7 patch match this template?"), the network asks three smaller questions in sequence, with non-linearities between them. Sequential non-linear questions are more expressive than one non-linear question with the same raw coverage.

4.3.4 Advantages

- Architectural simplicity: a single rule (3×3, same padding, stride 1) governs the entire conv stack.
- Strong feature extractor; VGG features transfer well to other tasks (detection, segmentation, style transfer).
- Systematic depth study published: VGG-11, 13, 16, 19 all show monotonic accuracy improvement with depth.

4.3.5 Disadvantages

- 138 million parameters (VGG-16): the largest portion is in the FC layers, not the conv stack.
- High memory consumption during training: all intermediate activations must be retained for backpropagation.
- Slow to train compared to later architectures with Batch Normalisation or residual connections.
- No significant architectural innovation beyond depth; later work (ResNet, Inception) showed that depth must be paired with other structural improvements (skip connections, parallel branches) to scale further.

The FC layer problem in VGGNet: VGG-16's three FC layers contain approximately 120 million of its 138 million parameters. These layers perform no spatial reasoning—they are a flat classifier bolted onto the conv stack. Global Average Pooling (GAP), used in later architectures, replaces FC layers entirely by averaging each feature map to a single value, cutting parameters dramatically with minimal accuracy loss.

4.4 Architecture Comparison Table

Property	LeNet-5	AlexNet	VGG-16	VGG-19
Year	1998	2012	2014	2014
Input size	$32 \times 32 \times 1$	$224 \times 224 \times 3$	$224 \times 224 \times 3$	$224 \times 224 \times 3$

Property	**LeNet-5**	**AlexNet**	**VGG-16**	**VGG-19**
Conv layers	3	5	13	16
FC layers	2	3	3	3
Total layers (trainable)	5	8	16	19
Filter sizes	5×5	11×11, 5×5, 3×3	3×3 only	3×3 only
Pooling	Avg 2×2	Max 3×3	Max 2×2	Max 2×2
Activation	tanh/sigmoid	ReLU	ReLU	ReLU
Regularisation	None	Dropout, data aug	Dropout	Dropout
Parameters	≈60K	≈60M	≈138M	≈144M
Dataset	MNIST	ImageNet	ImageNet	ImageNet
Key innovation	End-to-end conv net	ReLU, GPU, dropout	Uniform 3×3 depth	Deeper VGG-16
ILSVRC top-5 err	N/A	15.3%	7.3%	7.3%
Primary limitation	Too shallow for natural images	Irregular design, large filters	Massive parameter count	Even larger than VGG-16

Reading the parameter count: LeNet-5 has ≈60K parameters vs. AlexNet's ≈60M—a 1000× increase. VGG-16 adds another 2.3× beyond AlexNet. The explosion is driven primarily by FC layers, not the convolutional stack. Later architectures (ResNet, MobileNet) dramatically reduce this by removing or replacing FC layers with Global Average Pooling.